Programming with Python

BASIC CONCEPTS 2

Lesson Objectives

List

Tuples

Sets

Dictionaries

Working with files

List

SECTION 1

List

- **The important characteristics**
  - Lists are ordered.
  - Lists can contain any arbitrary objects.
  - List elements can be accessed by index.
  - Lists can be nested to arbitrary depth.
  - Lists are mutable.
  - Lists are dynamic.

```
a = ['foo', 'bar', 'baz', 'qux']
print(a)
```

```
['foo', 'bar', 'baz', 'qux']
```

List

Lists are ordered.

- A list is not merely a collection of objects
- It is an ordered collection of objects.
- The order in which you specify the elements when you define a list is an innate characteristic of that list and is maintained for that list's lifetime.

```
a = ['a', 'b', 'c', 'd']
b = ['b', 'a', 'd', 'c']
a == b
```

List

- **Lists Can Contain Arbitrary Objects.**
  - ✓A list can contain any assortment of objects

```
a = [21.42, 'foobar', 3, 4, 'bark', False, 3.14159]
```

  - ✓Lists can even contain complex objects, like functions, classes, and modules

```
def foo():
  pass

import numpy as np

list_a = [1, int, foo, np]
```

List

- List Elements Can Be Accessed by Index

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
print(a[0])
print(a[-1])
```

List

- **List Elements Can Be Accessed by Index**
  - ✓Slicing also works

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
print(a[0:2])
print(a[-3:-1])
```

List

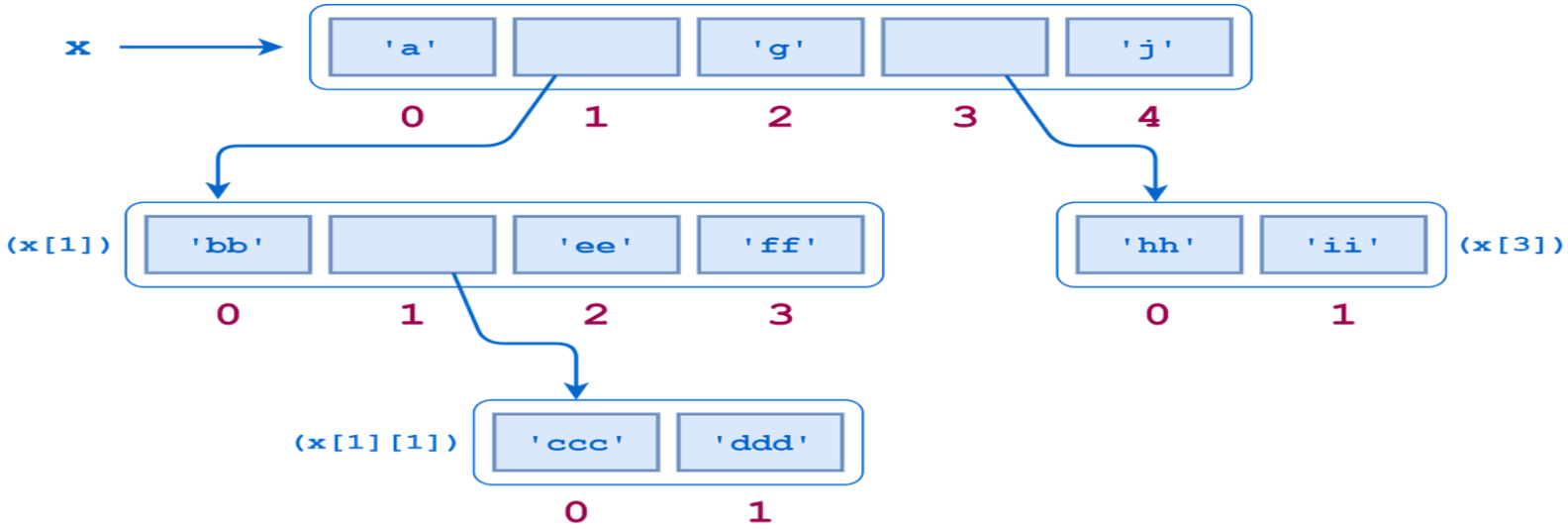
- **List Elements Can Be Accessed by Index**
  - ✓Other features of string slicing work analogously for list slicing as well

```
print(a[:4], a[0:4])
print(a[2:], a[2:len(a)])

a[:4] + a[4:]
a[:4] + a[4:] == a
```

List

- **Lists Can Be Nested**
  - ✓ A list can contain sublists



List

Lists Are Mutable

- ✓ Modifying a Single List Value

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
a[2] = 'x'
a[-2] = 10
```

- ✓ Modifying Multiple List Values

```
a = ['foo', 'bar', 'baz', 'qux', 'quux', 'corge']
print(a[1:4])
a[1:4] = [0.5, 0.1, 55, 33, 231, 1213]
print(a)
```

List

- **Methods That Modify a List**
  - ✓ a.append(<obj>)
  - ✓ a.extend(<iterable>)
  - ✓ a.insert(<index>, <obj>)
  - ✓ a.remove(<obj>)
  - ✓ a.pop(index=-1)

Tuples

SECTION 2

Tuples

Defining and Using Tuples

- Tuples are defined by enclosing the elements in parentheses (()) instead of square brackets ([]).
- Tuples are immutable.

```
te = ()
t = ('fpt', 1, 4)
```

Tuples

- **Why use a tuple instead of a list?**
  - ✓Program execution is faster when manipulating a tuple than it is for the equivalent list
  - ✓Sometimes you don’t want data to be modified

```
te = ()
t = ('fpt', 1, 4)
```

Set

SECTION 3

Set

- A set is a collection which is unordered and unindexed

  Note: the set list is unordered, meaning: the items will appear in a random order.

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
print(thisset)
```

Set

- You cannot access items in a set by referring to an index or a key

using a for loop

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
for x in thisset:
   print(x)
```

using the in keyword.

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
print("kiwi" in thisset)
```

Dictionary

Built-in Set Methods

- Add Items

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
thisset.add("orange")
print(thisset)
```

- Update Items

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
thisset.update(["orange", "mango", "grapes", "kiwi"])
print(thisset)
```

Dictionary

Built-in Set Methods

✓Remove Item with ***.remove()***

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
thisset.remove("banana")
print(thisset)
```

✓Update Items with ***.discard()***

```
thisset = {"banana", "apple", "cherry", 'kiwi', 'banana', 'apple'}
thisset.discard("banana")
print(thisset)
```

.remove() ? .discard()

Dictionary

SECTION 4

Dictionary

- **Dictionaries and lists share the following characteristics**
  - ✓ Both are mutable.
  - ✓ Both are dynamic. They can grow and shrink as needed.
  - ✓ Both can be nested. A list can contain another list. A dictionary can contain another dictionary. A dictionary can also contain a list, and vice versa.

Dictionary

- Dictionaries differ from lists primarily in how elements are accessed:
  - ✓ List elements are accessed by their position in the list, via indexing.
  - ✓ Dictionary elements are accessed via keys.

Dictionary

- **Defining a Dictionary**

```
d = {
    <key>: <value>,
    <key>: <value>,
      .
      .
      .
    <key>: <value>
}
```

```
d = dict([
    (<key>, <value>),
    (<key>, <value),
      .
      .
      .
    (<key>, <value>)
])
```

Dictionary

- **Accessing Dictionary Values**
  - ✓Access by key

```
thisdict[<key>]
```

  - ✓Modify by key

```
thisdict[<key>] = <new value>
```

Dictionary

- **Building a Dictionary Incrementally**

```
person = {}
print(type(person))
# <class 'dict'>

person['fname'] = 'Joe'
person['lname'] = 'Fonebone'
person['age'] = 51
person['spouse'] = 'Edna'
person['children'] = ['Ralph', 'Betty', 'Joey']
person['pets'] = {'dog': 'Fido', 'cat': 'Sox'}
```

Dictionary

- **Operators and Built-in Functions**

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print('brand' in thisdict)
print('uuu' in thisdict)
print('uuu' not in thisdict)
```

Dictionary

Built-in Dictionary Methods

- d.clear()
- d.get(<key>[, <default>])
- d.items()
- d.keys()
- d.values()
- d.pop(<key>[, <default>])
- d.popitem()
- d.update(<obj>)

Thank you